

Ph. D. Juan David Martínez Vargas
Ph. D. Raul Andrés Castañeda

Escuela de Ciencias Aplicadas e Ingeniería

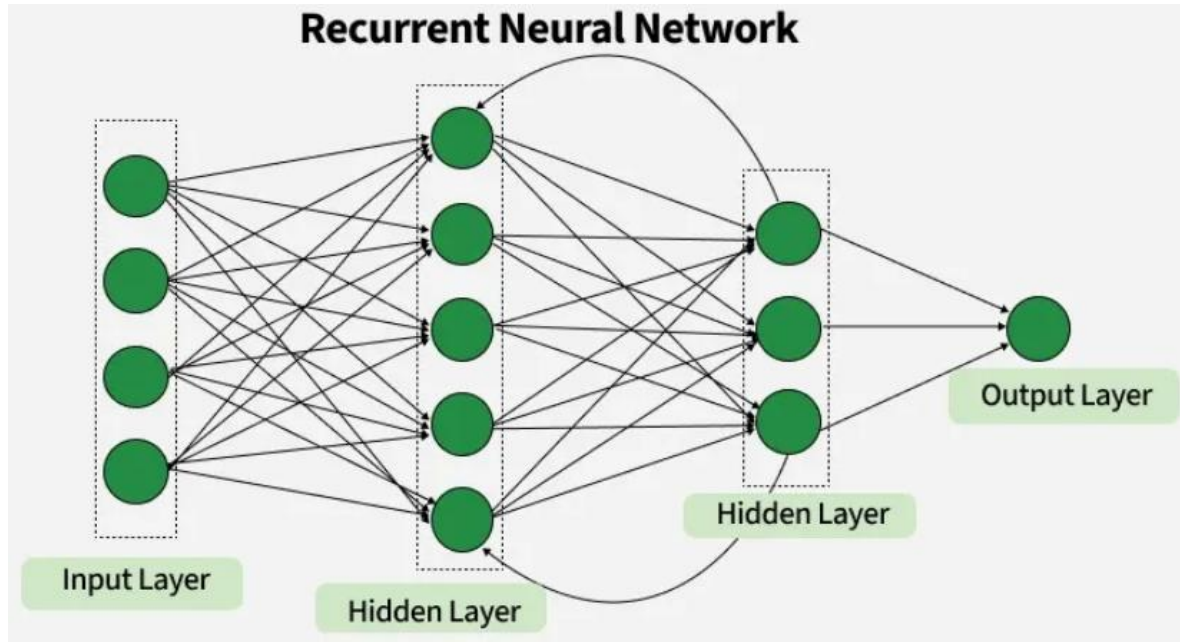
Lecture 11

Recurrent Neural Networks I (RNN)

Agenda

- **¿Qué son las redes neuronales Recurrentes RNN?**
- **Por qué son importantes.**
- **Diferentes formas de modelar texto (secuencias)**
- **Modelando secuencias con RNNs**
- **Diferentes tipos de tareas de modelado de secuencias**
- **Many to one RNNs**
- **Ejemplos**

RNN - Introducción



¿Qué pasa si el orden de los datos importa?

Es importante que la red se retroalimente a sí misma

Predicción de series temporales: RNN destacan en tareas de predicción, como la cotización de la bolsa y la previsión meteorológica.

Procesamiento del lenguaje natural (PLN): RNN son fundamentales en tareas de PLN como el modelado del lenguaje, el análisis de sentimientos y la traducción automática.

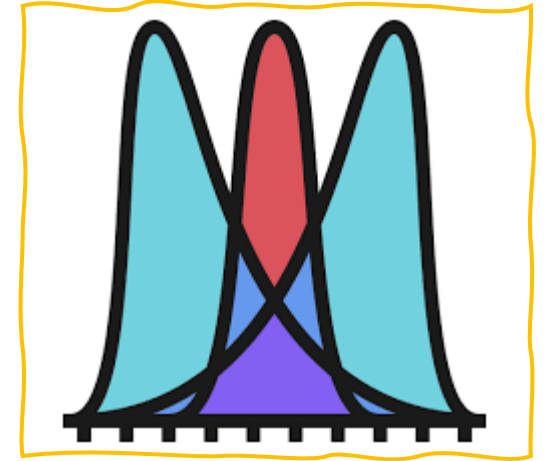
Reconocimiento de voz: Las RNN capturan patrones temporales en los datos de voz, lo que facilita la conversión de voz a texto y otras aplicaciones relacionadas con el audio.

Procesamiento de imágenes y vídeo: Combinadas con capas convolucionales, las RNN ayudan a analizar secuencias de vídeo, expresiones faciales y reconocimiento de gestos.

RNN Introducción

Imagina leer una oración e intentar predecir la siguiente palabra; no te basas solo en la palabra actual, sino que también recuerdas las anteriores.

Las RNN funcionan de manera similar, "recordando" información pasada; es decir, consideran todas las palabras anteriores para elegir la siguiente palabra.



¿Cuál es el dato más probable?

RNN Introducción

"Raw" training dataset

$\mathbf{x}^{[1]}$ = "The sun is shining"

$\mathbf{x}^{[2]}$ = "The weather is sweet"

$\mathbf{x}^{[3]}$ = "The sun is shining,
the weather is sweet, and
one and one is two"

$\mathbf{y} = [0, 1, 0]$

class labels

vocabulary = {
'and': 0,
'is': 1
'one': 2,
'shining': 3,
'sun': 4,
'sweet': 5,
'the': 6,
'two': 7,
'weather': 8,
}

Bag of Words (BoW)

Training set as design matrix

$$\mathbf{X} = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 2 & 3 & 2 & 1 & 1 & 1 & 2 & 1 & 1 \end{bmatrix}$$

$\mathbf{y} = [0, 1, 0]$

class labels

training

Classifier

(e.g., logistic regression, MLP, ...)

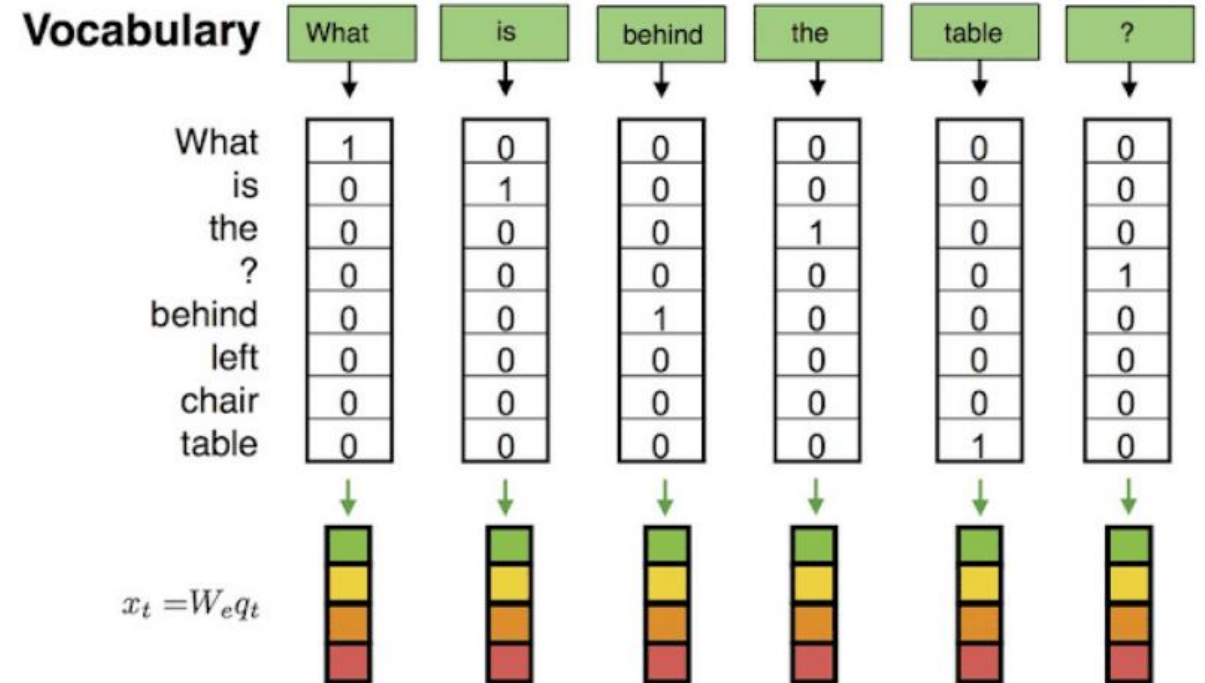
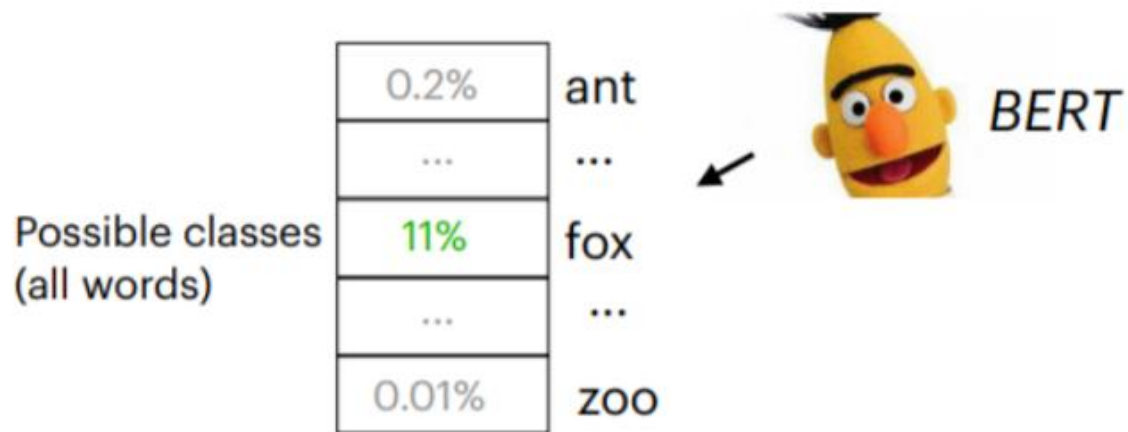
Esta representación es útil... pero ignora algo fundamental:
el orden.

Bag of Words (BoW)

- Los algoritmos DL no pueden trabajar directamente con texto sin procesar; el texto debe convertirse en vectores de números.

El lenguaje depende del contexto

BoW reconoce qué palabras hay
Pero NO sabe en qué orden están



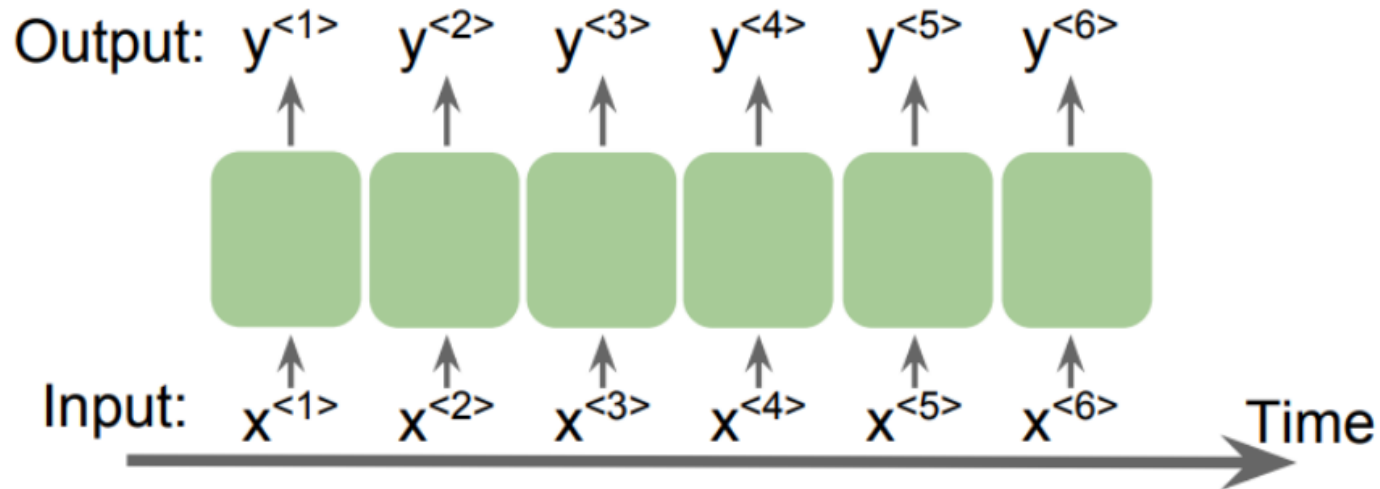
Sentencia de entrada: A quick brown fox jumps over the lazy dog

15% enmascarado al azar: A quick brown [MASK] jumps over the lazy dog

RNN Introducción

Datos de secuencia: el orden importa

- La película que mi amigo **no** ha visto es buena
- La película que mi amigo ha visto **no** es buena



BoW No sabe:

- quién va primero
- qué depende de qué

$x^{<t>}$ = vector de la palabra en el tiempo t

Fuente de la imange: Sebastian Raschka, Vahid Mirjalili. Python Machine Learning. 3rd Edition. Packt, 2019

Embedding

¿Cómo lograr que un computador entienda el significado de las palabras?

Un **embedding** es una función que asigna a cada elemento discreto (por ejemplo, una palabra) un vector en un espacio continuo de dimensión baja, de tal forma que relaciones semánticas o estructurales se preservan en ese espacio.

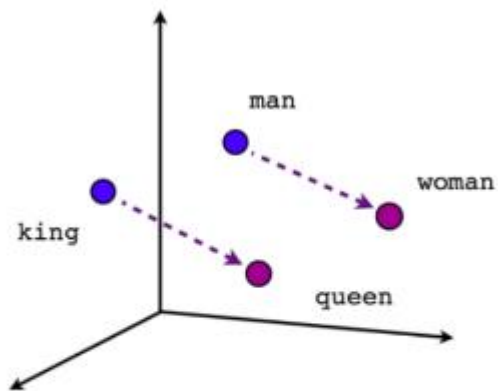


Embedding

Imagina un espacio de embeddings de tres dimensiones donde cada eje representa un color primario: rojo, verde y azul.

La palabra “cielo” podría representarse como el vector $(0, 0, 1)$ porque está asociada con el color azul.

En cambio, “hierba” podría representarse como $(0, 1, 0)$ porque está asociada con el color verde.

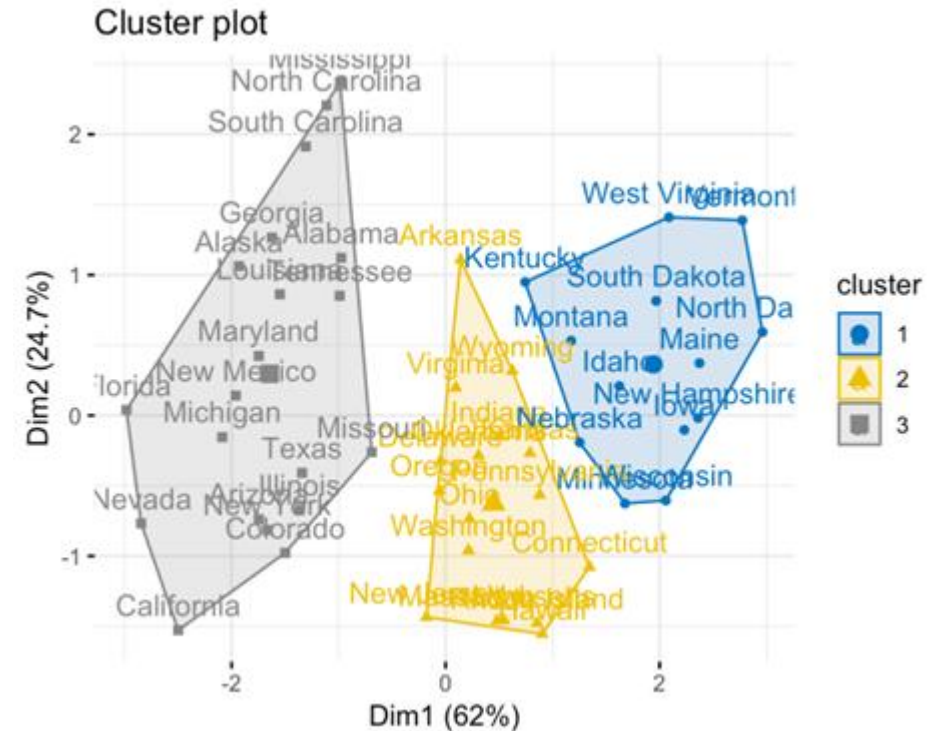


Male-Female

$$E(dog) = [0.2, -0.1, 0.7]$$

$$E(car) = [-0.9, 0.3, 0.1]$$

$$E(cat) = [0.2, -0.1, 0.7]$$



"the cat eats fish"

→ ["the", "cat", "eats", "fish"]

La **tokenización** es el proceso de convertir un texto en una secuencia de unidades más pequeñas llamadas *tokens*, que pueden ser palabras, subpalabras o caracteres.

→
 $x^{<1>} = E("the")$
 $x^{<2>} = E("cat")$
 $x^{<3>} = E("eats")$
 $x^{<4>} = E("fish")$

Embeddings

→ $x^{<1>} \rightarrow x^{<2>} \rightarrow x^{<3>} \rightarrow x^{<4>}$ **RNN procesa secuencia**

PERO CUIDADO Las RNN no son solo para lenguaje... son para cualquier dato donde el orden importa

Trabajar con datos secuenciales

- Clasificación de texto
- Reconocimiento de voz (modelado acústico)
- Traducción de idiomas
- Modelado de secuencias de ADN o (aminoácidos / proteínas)

Stock market predictions

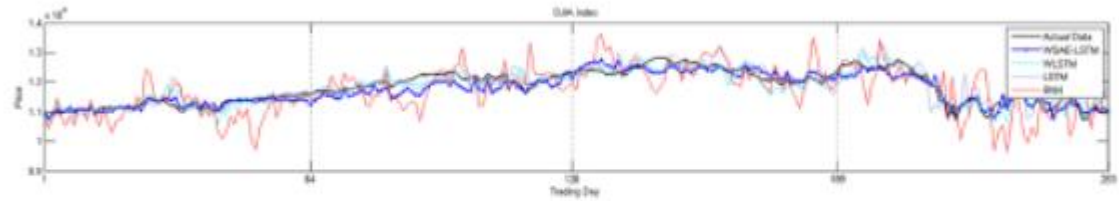
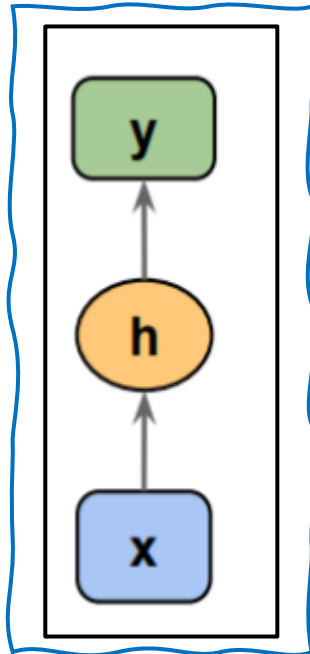


Fig 8. Displays the actual data and the predicted data from the four models for each stock index in Year 1 from 2010.10.01 to 2011.09.30.

<https://doi.org/10.1371/journal.pone.0180944.g008>

El interior de una RNN

Feedforward
DL model



RNN model

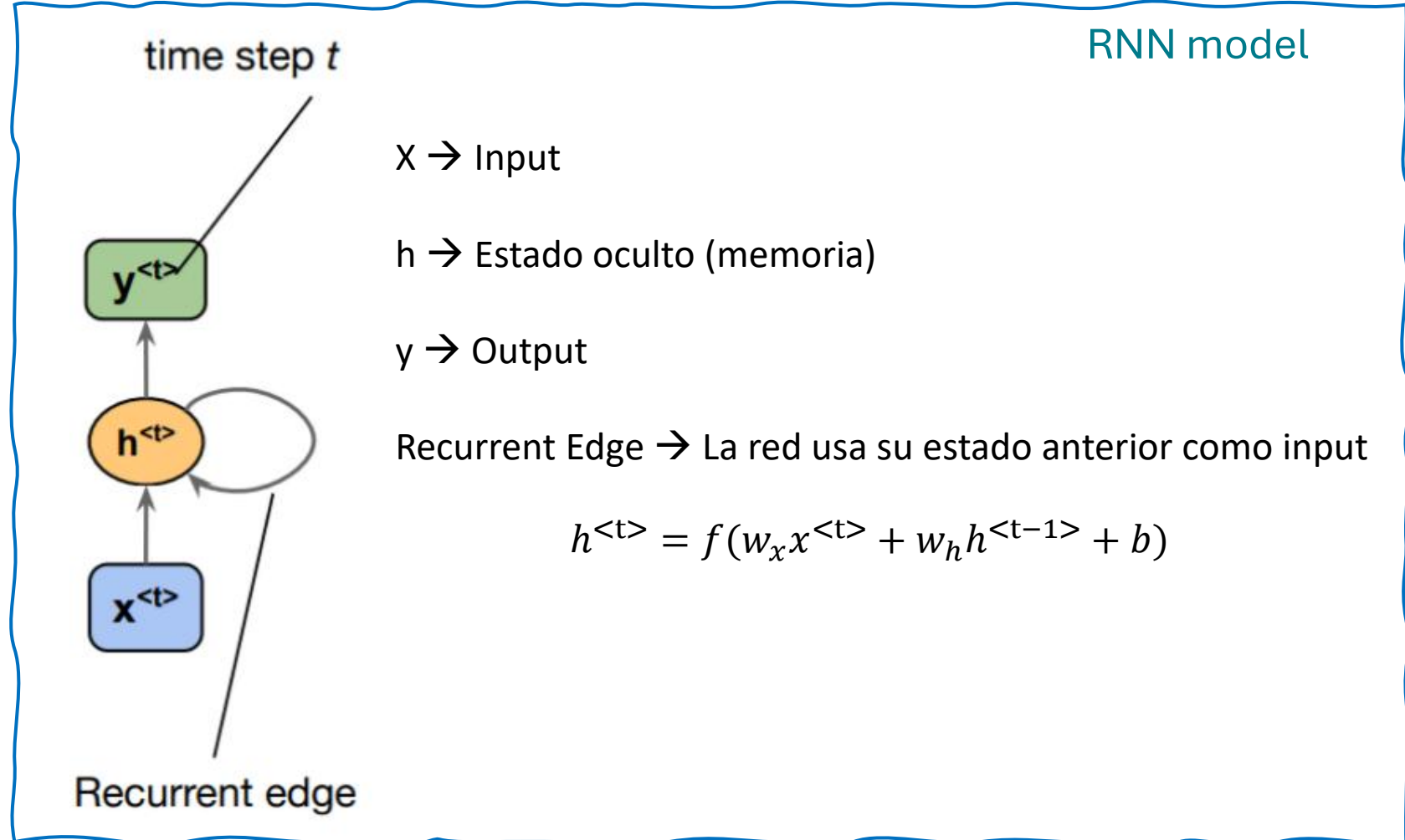
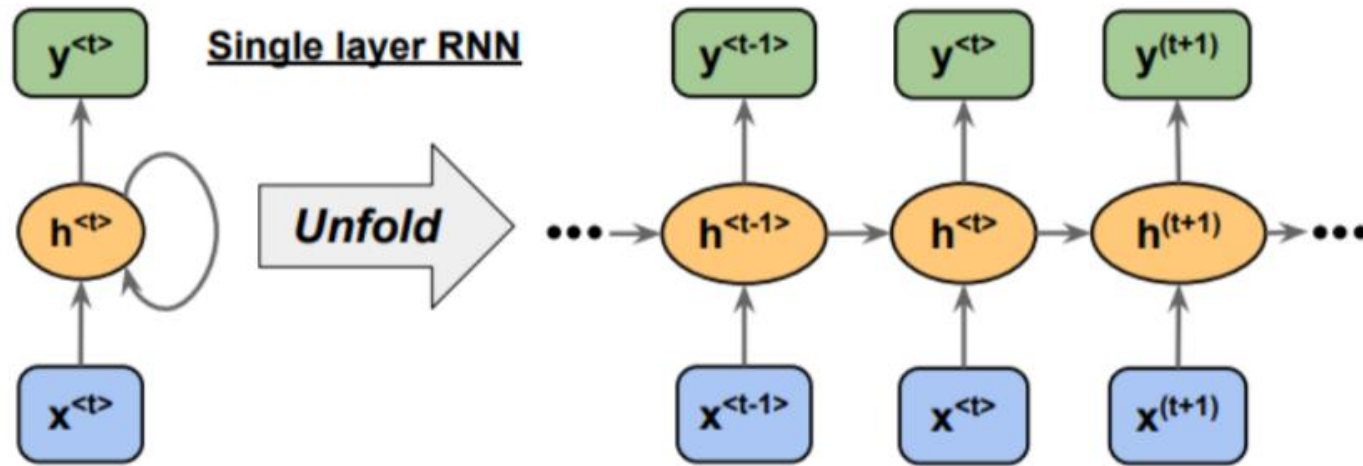


Image source: Sebastian Raschka, Vahid Mirjalili. Python Machine Learning. 3rd Edition. Packt, 2019

El interior de una RNN



La red se “desenrolla” en el tiempo:

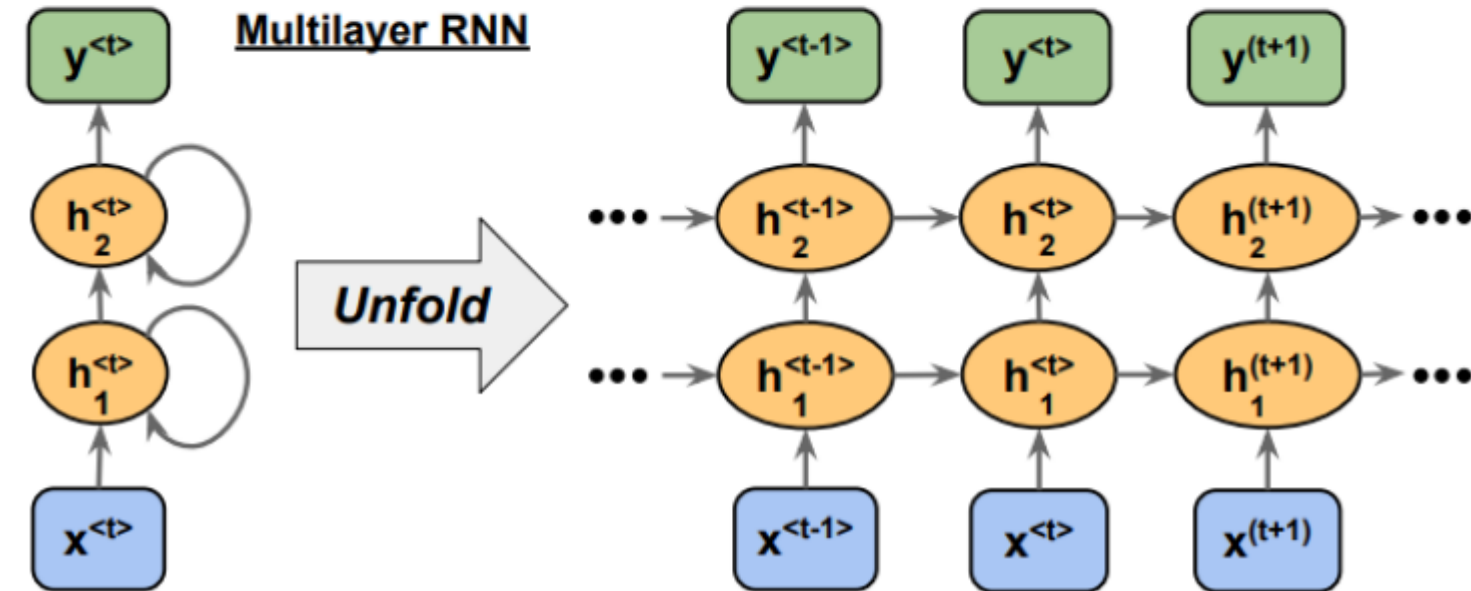
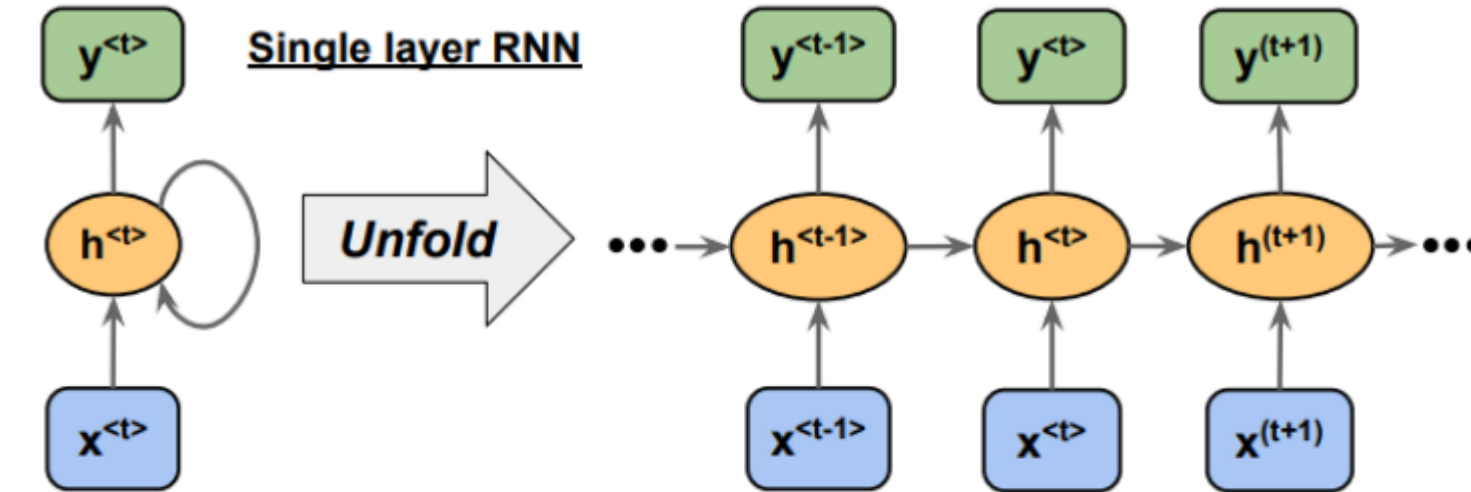
El unfolding de una RNN es el proceso de expandir la estructura recurrente a lo largo de los pasos de tiempo.

Durante el despliegue, cada paso de la secuencia se representa como una capa separada en una serie que ilustra cómo fluye la información a través de cada paso de tiempo.

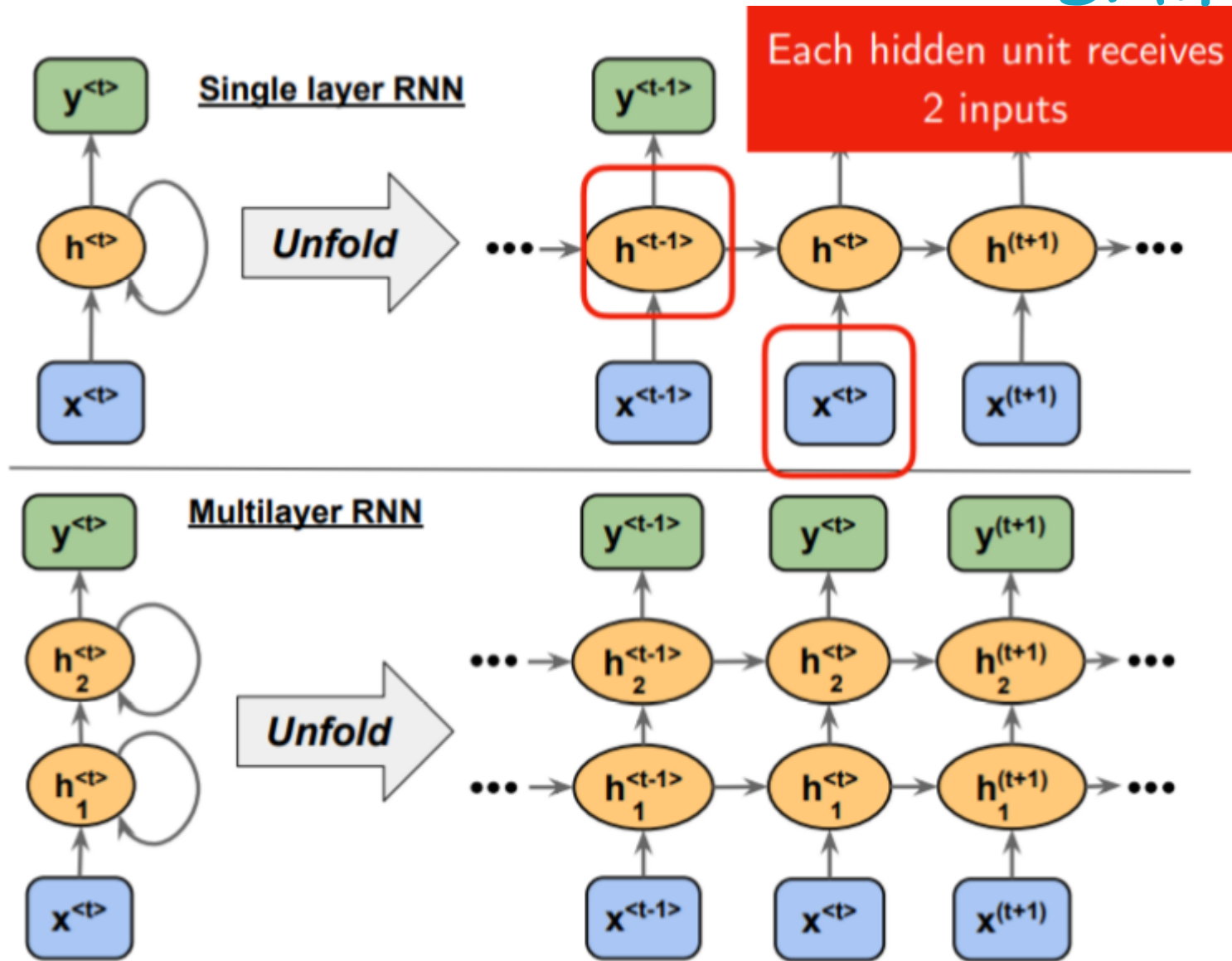
backpropagation through time (BPTT) proceso de aprendizaje en el que los errores se propagan a través de los pasos de tiempo.

Image source: Sebastian Raschka, Vahid Mirjalili. Python Machine Learning. 3rd Edition. Packt, 2019

El interior de una RNN



El interior de una RNN



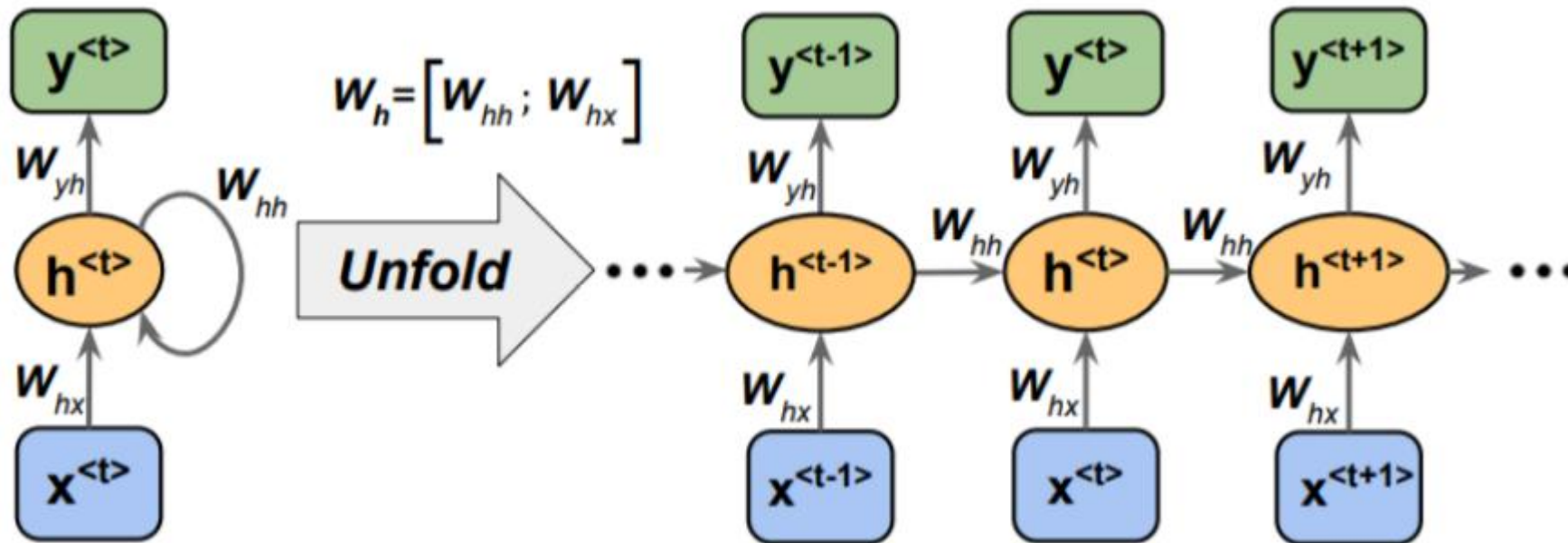
El interior de una RNN

$$h^{<t>} = f(w_x x^{<t>} + w_h h^{<t-1>} + b)$$

$w_x \rightarrow$ Pesos de la entrada

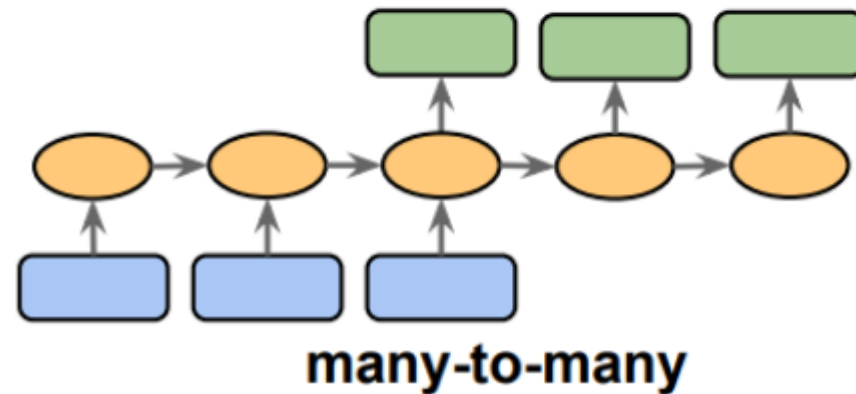
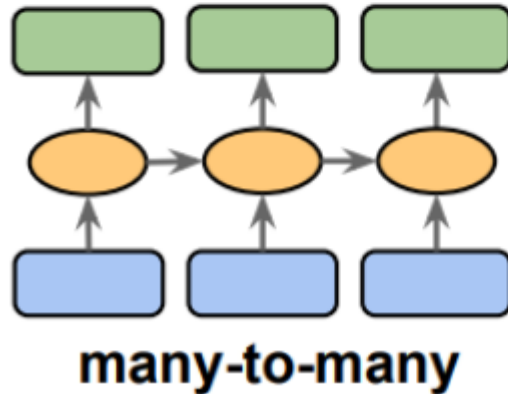
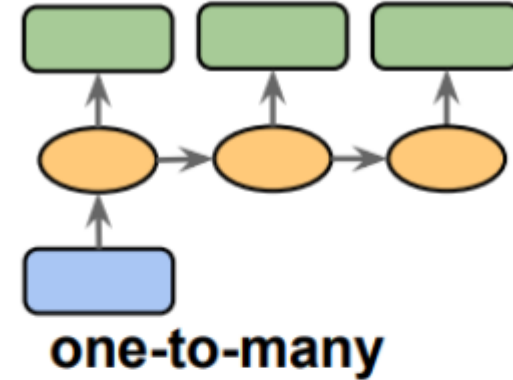
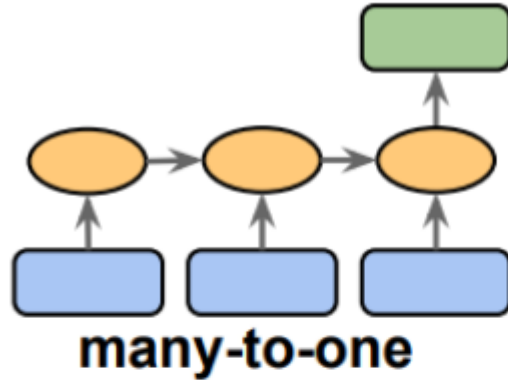
$w_h \rightarrow$ Pesos de la memoria

$$h^{<t>} = f(\underbrace{w_x x^{<t>}}_{\text{Presente}} + \underbrace{w_h h^{<t-1>}}_{\text{Pasado}} + b)$$

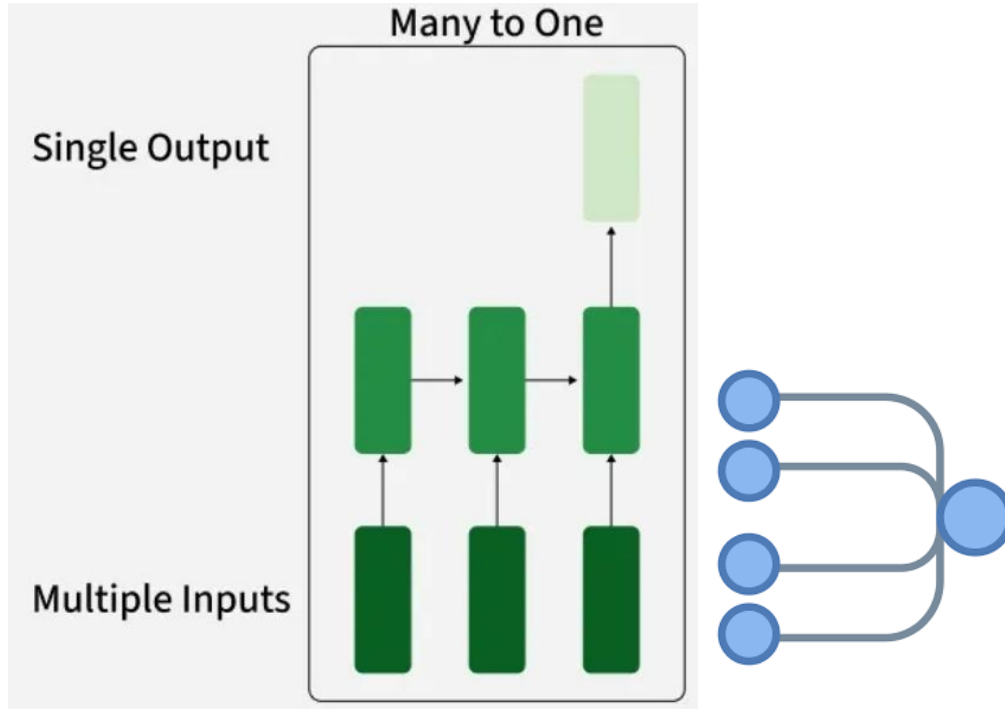


Tipos de RNN

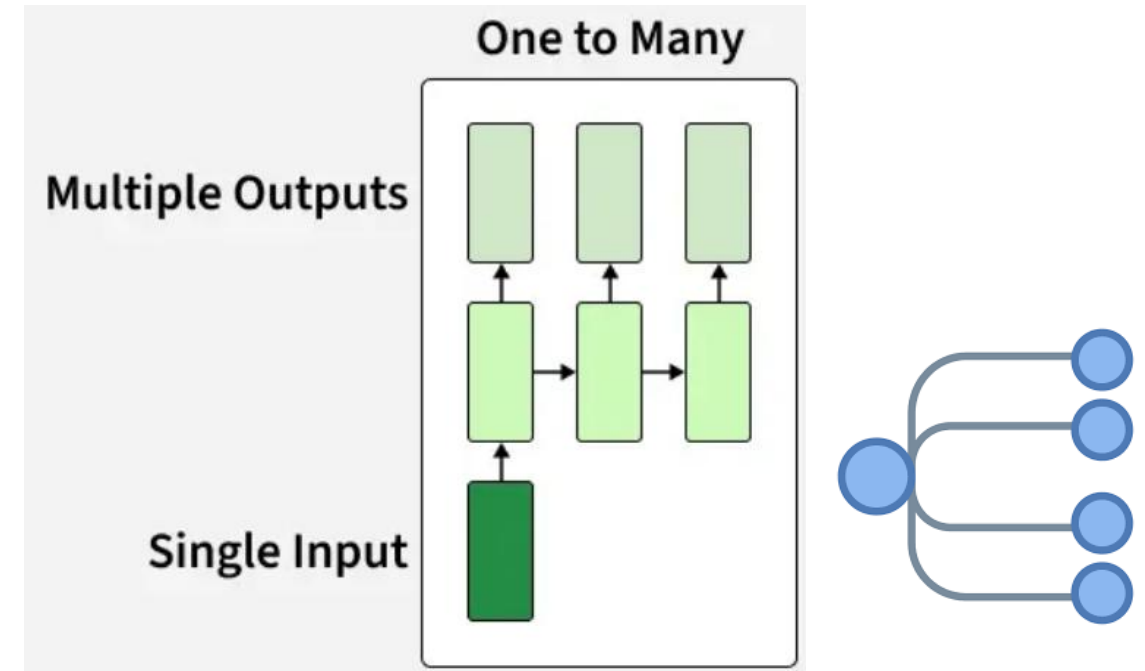
Existen cuatro tipos de RNN según el número de entradas y salidas de la red



Tipos de RNN

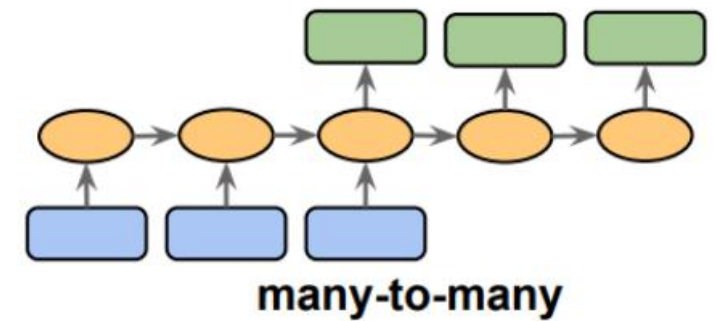
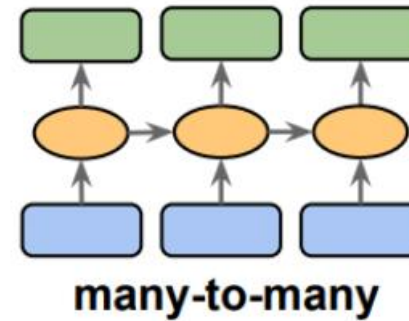
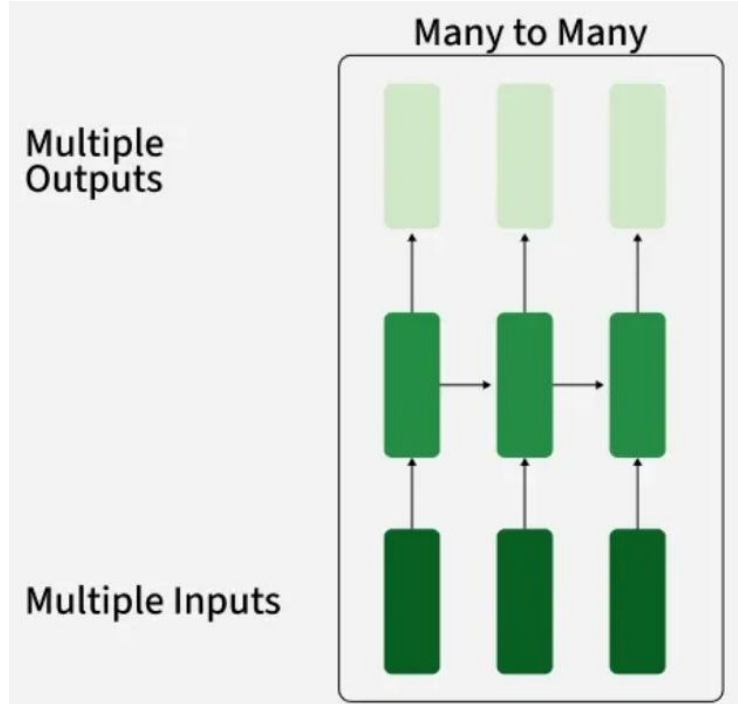


Many to one: Los datos de entrada son una secuencia, pero la salida es un vector de tamaño fijo, no una secuencia. **Ejemplo:** Análisis de sentimiento, la entrada es un texto y la salida es una etiqueta de clase.



One to many: Los datos de entrada están en un formato estándar (no una secuencia), la salida es una secuencia. **Ejemplo:** Subtítulos de imágenes, donde la entrada es una imagen, la salida es una descripción de texto de esa imagen

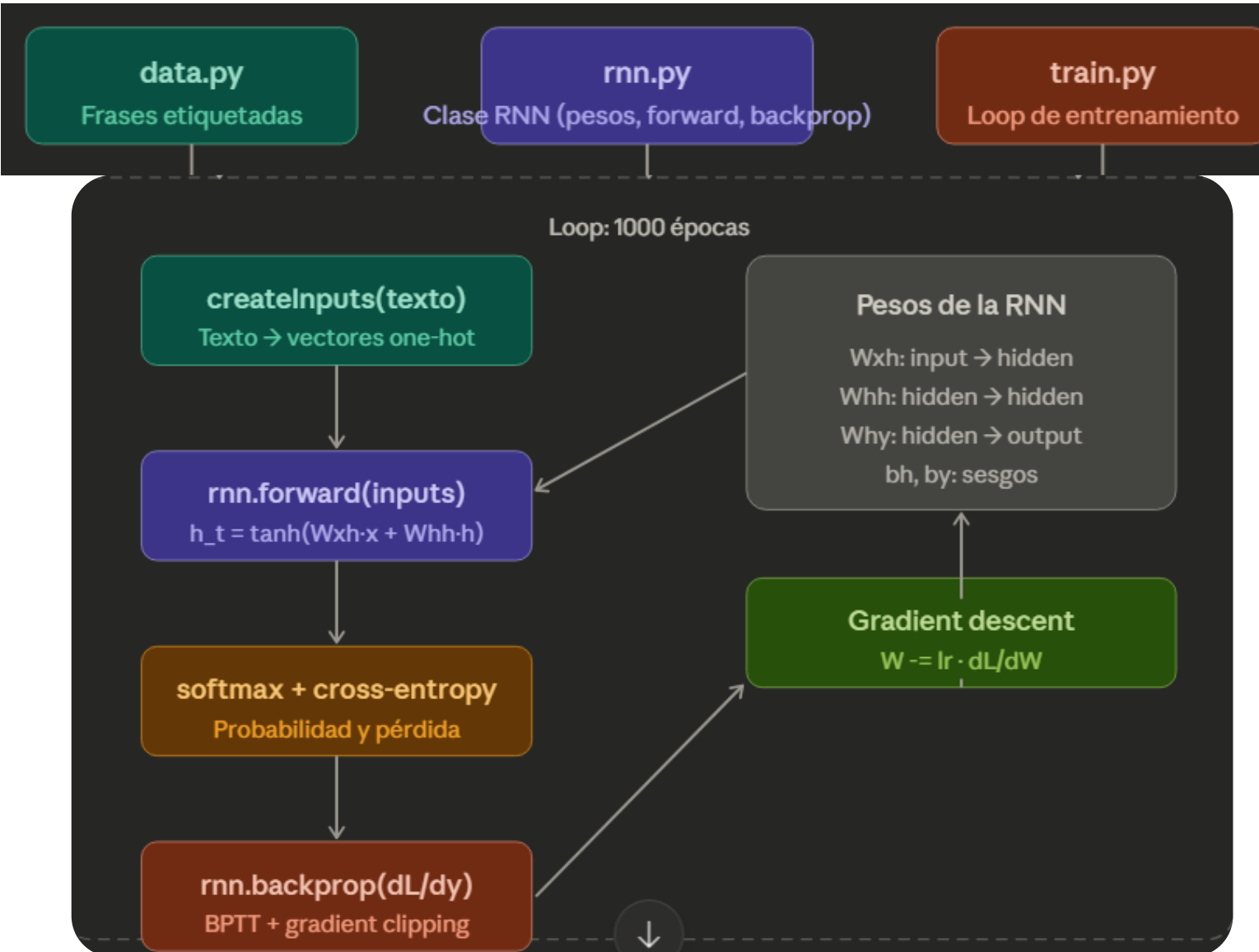
Tipos de RNN



Many to many: Tanto las entradas como las salidas son secuencias. Puede ser directo o retrasado. **Ejemplo:** Subtítulos de video, es decir, describir una secuencia de imágenes a través de texto (directo). Traducir un idioma a otro (retrasado)

RNN desde cero

(RNN) desde cero para realizar una tarea de análisis de sentimientos



Zhou, V. (n.d.). GitHub profile. GitHub.
<https://github.com/vzhou842>

RNN desde cero

```
train_data = {  
    'good': True,  
    'bad': False,  
    'happy': True,  
    'sad': False,  
    'not good': False,  
    'not bad': True,  
    'not happy': False,  
    'not sad': True,  
    'very good': True,  
    'very bad': False,  
    'very happy': True,  
    'very sad': False,  
    'i am happy': True,
```

```
'this is good': True,  
'i am bad': False,  
'this is bad': False,  
'i am sad': False,  
'this is sad': False,  
'i am not happy': False,  
'this is not good': False,  
'i am not bad': True,  
'this is not sad': True,  
'i am very happy': True,  
'this is very good': True,  
'i am very bad': False,  
'this is very sad': False,  
'this is very happy': True,  
'i am good not bad': True,  
'this is good not bad': True,  
'i am bad not good': False,  
'i am good and happy': True,  
'this is not good and not happy': False,
```

```
test_data = {  
    'this is happy': True,  
    'i am good': True,  
    'this is not happy': False,  
    'i am not good': False,  
    'this is not bad': True,  
    'i am not sad': True,  
    'i am very good': True,  
    'this is very bad': False,  
    'i am very sad': False,  
    'this is bad not good': False,  
    'this is good and happy': True,  
    'i am not good and not happy': False,  
    'i am not at all sad': True,  
    'this is not at all good': False,  
    'this is not at all bad': True,  
    'this is good right now': True,  
    'this is sad right now': False,  
    'this is very bad right now': False,  
    'this was good earlier': True,  
    'i was not happy and not good earlier': False,  
}
```

RNN desde cero

```
vocab = list(set([w for text in train_data.keys() for w in text.split(' ')]))  
vocab_size = len(vocab)
```

'i am happy' → 'i', 'am', 'happy'

'not happy' → 'not', 'happy'

w = ['i', 'am', 'happy', 'not', 'happy']

18 unique words found

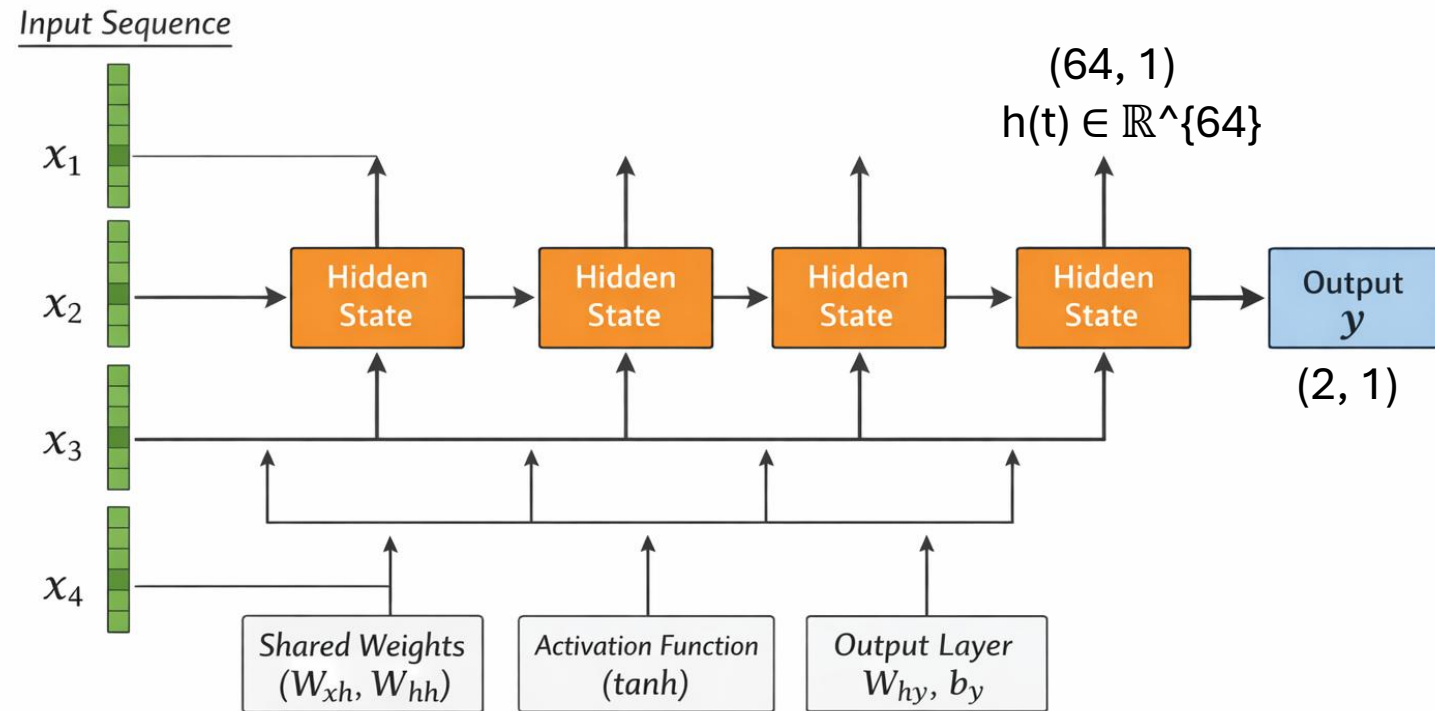
```
# Assign indices to each word.  
word_to_idx = { w: i for i, w in enumerate(vocab) }  
idx_to_word = { i: w for i, w in enumerate(vocab) }  
  
for idx in sorted(idx_to_word.keys()):  
    print(f'{idx} -> {idx_to_word[idx]}')
```

0 -> am
1 -> i
2 -> or
3 -> and
4 -> is
5 -> not
6 -> this
7 -> all
8 -> sad
9 -> bad

Word	Idx	One-Hot (first 10 positions)
am	0	[1 0 0 0 0 0 0 0 0 0 ...]
i	1	[0 1 0 0 0 0 0 0 0 0 ...]
or	2	[0 0 1 0 0 0 0 0 0 0 ...]
and	3	[0 0 0 1 0 0 0 0 0 0 ...]
is	4	[0 0 0 0 1 0 0 0 0 0 ...]
not	5	[0 0 0 0 0 1 0 0 0 0 ...]
this	6	[0 0 0 0 0 0 1 0 0 0 ...]
all	7	[0 0 0 0 0 0 0 1 0 0 ...]
sad	8	[0 0 0 0 0 0 0 0 1 0 ...]
bad	9	[0 0 0 0 0 0 0 0 0 1 ...]

RNN desde cero

One-Hot (first 10 positions)														
[1	0	0	0	0	0	0	0	0	0	0	...			
[0	1	0	0	0	0	0	0	0	0	0	...			
[0	0	1	0	0	0	0	0	0	0	0	...			
[0	0	0	1	0	0	0	0	0	0	0	...			
[0	0	0	0	1	0	0	0	0	0	0	...			
[0	0	0	0	0	1	0	0	0	0	0	...			
[0	0	0	0	0	0	1	0	0	0	0	...			
[0	0	0	0	0	0	0	1	0	0	0	...			
[0	0	0	0	0	0	0	0	1	0	0	...			
[0	0	0	0	0	0	0	0	0	1	0	...			



$h(1) \rightarrow$ entiende "I"
 $h(2) \rightarrow$ entiende "I love"
 $h(3) \rightarrow$ entiende "I love this"
 $h(4) \rightarrow$ entiende "I love this movie"

$x_1 \rightarrow h_1$
 $x_2 + h_1 \rightarrow h_2$
 $x_3 + h_2 \rightarrow h_3$
 $x_4 + h_3 \rightarrow h_4 \rightarrow y$

RNN desde cero

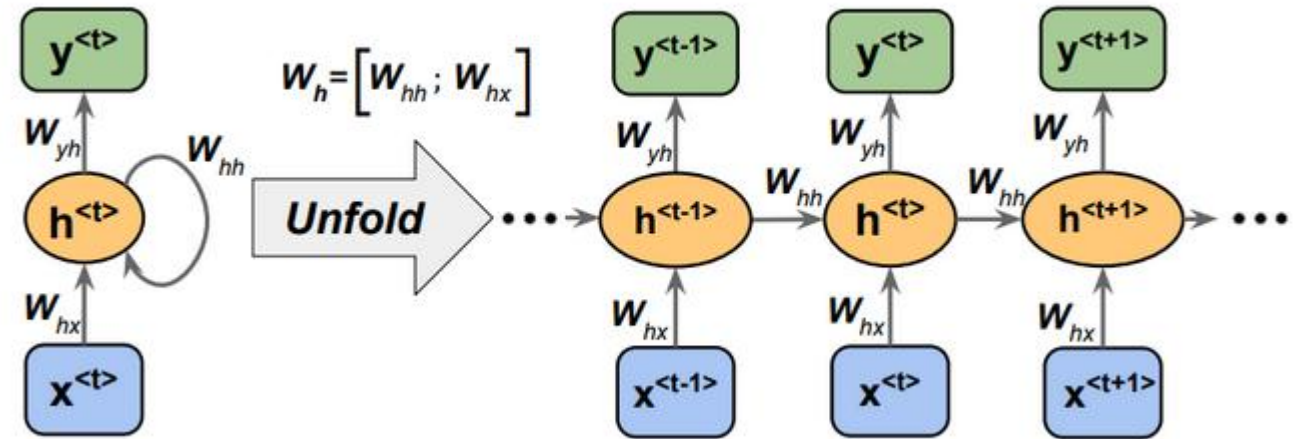
$$h^{<t>} = \tanh(w_{xh}x^{<t>} + w_{hh}h^{<t-1>} + b_h)$$

$$y = W_{yh}h^{<t>} + b_y$$

W_{hx} : codifica la información de la entrada actual

W_{hh} : mantiene y actualiza el contexto temporal

W_{yh} : transforma el contexto final en una predicción



```
def __init__(self, input_size, output_size, hidden_size=64):  
    # Weights  
    self.Whh = randn(hidden_size, hidden_size) / 1000  
    self.Wxh = randn(hidden_size, input_size) / 1000  
    self.Wyh = randn(output_size, hidden_size) / 1000  
  
    # Biases  
    self.bh = np.zeros((hidden_size, 1))  
    self.by = np.zeros((output_size, 1))
```

RNN desde cero

```
def processData(data, backprop=True):
    items = list(data.items())
    random.shuffle(items)

    loss = 0
    num_correct = 0

    for x, y in items:
        inputs = createInputs(x)
        target = int(y)

        # Forward
        out, _ = rnn.forward(inputs)
        probs = softmax(out)

        # Calculate loss / accuracy
        loss -= np.log(probs[target].item())
        num_correct += int(np.argmax(probs) == target)

    if backprop:
        # Build dL/dy
        d_L_d_y = probs
        d_L_d_y[target] -= 1

        # Backward
        rnn.backprop(d_L_d_y)

    return loss / len(data), num_correct / len(data)
```

Mezcla los datos para evita que el modelo aprenda orden fijo

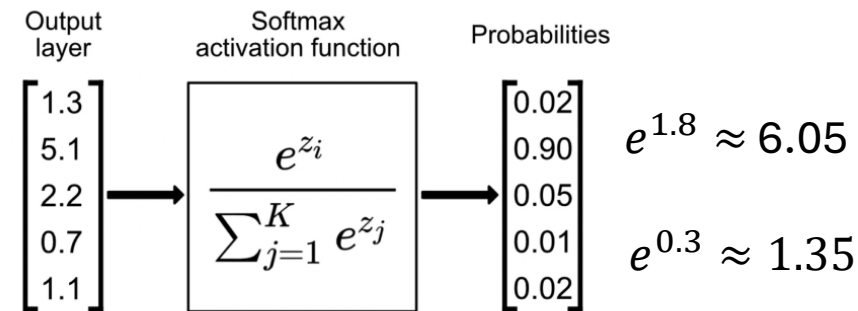
x = texto

y = etiqueta (True/False)

Convertir texto → vectores

"I love this" → [x1, x2, x3]

out = [[1.8],[0.3]]



$$\text{softmax}(1.8) = \frac{6.06}{7.40} \approx 0.82$$
$$\text{softmax}(0.3) = \frac{1.35}{7.40} \approx 0.18$$

probs = [[0.82],[0.18]]

RNN desde cero

```
def processData(data, backprop=True):
    items = list(data.items())
    random.shuffle(items)

    loss = 0
    num_correct = 0

    for x, y in items:
        inputs = createInputs(x)
        target = int(y)

        # Forward
        out, _ = rnn.forward(inputs)
        probs = softmax(out)

        # Calculate Loss / accuracy
        loss -= np.log(probs[target].item())
        num_correct += int(np.argmax(probs) == target)

        if backprop:
            # Build dL/dy
            d_L_d_y = probs
            d_L_d_y[target] -= 1

            # Backward
            rnn.backprop(d_L_d_y)

    return loss / len(data), num_correct / len(data)
```

probs = [[0.82],[0.18]]

target = 1

probs[target] = probs[1] = 0.18

$-\log(0.18) \approx 1.71$

si probs[target] es pequeña, la pérdida es grande

[0.82, 0.18] → predicción = 0

target = 0

probs[target] = probs[0] = 0.82

$-\log(0.18) \approx 0.19$

si probs[target] es grande, la pérdida es pequeña

RNN desde cero

```
predict("this is good")
predict("this is bad")
predict("i love this movie")
predict("i hate this")
```

```
Text: "this is good"
Probabilities: [4.45345507e-06 9.99995547e-01]
Predicted class: 1
Confidence: 1.0000
Sentiment: Positive
```

```
Text: "this is bad"
Probabilities: [9.99495644e-01 5.04355895e-04]
Predicted class: 0
Confidence: 0.9995
Sentiment: Negative
```

```
Traceback (most recent call last):
  File "/teamspace/studios/this_studio/main.py", line 145, in <module>
    predict("i love this movie")
  File "/teamspace/studios/this_studio/main.py", line 123, in predict
    inputs = createInputs(text)
              ~~~~~
  File "/teamspace/studios/this_studio/main.py", line 38, in createInputs
    v[word_to_idx[w]] = 1
      ~~~~~
KeyError: 'love'
```

El modelo tiene un **vocabulario cerrado**

No puede manejar palabras nuevas

Un modelo basado en one-hot solo puede procesar palabras que vio durante el entrenamiento.

RNN desde cero

Código manual	PyTorch	
<code>self.Wxh, self.Whh, self.bh</code>	<code>nn.RNN(input_size, hidden_size)</code>	PyTorch agrupa esos pesos internamente
<code>self.Why, self.by</code>	<code>nn.Linear(hidden_size, 2)</code>	Es simplemente una capa lineal
<code>rnn.backprop(d_L_d_y)</code>	<code>loss.backward()</code>	Autograd calcula todos los gradientes
<code>np.clip(d, -1, 1)</code>	<code>clip_grad_norm_(params, 1.0)</code>	Mismo concepto, mejor implementado
<code>W -= lr * dW</code>	<code>optimizer.step()</code>	El optimizer maneja la actualización

El accuracy del 100% es engañoso

El modelo memorizó las frases de entrenamiento, no aprendió el lenguaje.

La RNN no entiende semántica, solo patrones estadísticos

Para el modelo, "not very happy" es una secuencia de 4 vectores. No sabe que "not" niega a "very happy". Solo sabe que cuando vio "very happy" antes, era positivo, y ese patrón domina.

El dataset está desbalanceado en este caso específico

Frases con "not very" en entrenamiento:

- "i am not very happy" → no existe
- "very" casi siempre aparece en frases positivas en el train set

- Tome como referencia el proyecto “**RNN from Scratch**” y, a partir del archivo `rnn.py`, describa matemáticamente y con sus propias palabras qué ocurre en los métodos `forward` y `backprop`. Es decir, explique el modelo matemático de una red neuronal recurrente (RNN).

Esta información debería ser añadida en el próximo laboratorio como un apéndice

- Tome el proyecto de referencia “**RNN from Scratch**” y modifique en el archivo `main.py` la representación **one-hot encoding** por una representación basada en **embeddings**. Ejecute nuevamente el código y analice si existe alguna diferencia en el comportamiento o en los resultados del modelo.
- Tome el archivo `L10_sentimientos_rnn.ipynb` y ejecútelo después de incorporar un **embedding**. Además, realice el ejercicio de ejecución **step by step**. A partir de ello, responda: ¿cómo podría evitar la inconsistencia que aparece en la siguiente frase?

"i am not very happy"

→ POSITIVO ✓ (neg: 0.0% | pos: 100.0%)